

The ptkdb debugger implements a DB package, to access the DB internals and a Devel::ptkdb package. There are subs scattered across both packages and new variables introduced in the DB package, such as `$DB::on = 0; $DB::subroutine_depth = 0; # our subroutine depth counter $DB::step_over_depth = -1;` In general, I would think most, if not all, of the methods should belong in the Devel::ptkdb. In particular this statement is confusing: `$DB::window = new Devel::ptkdb;` It looks as if the code adds the whole graphical environment to the Debugger environment. What SHOULD be the breakdown for what functions go in what package?

I'll separate the answer into "what Perl requires in `DB`" versus "what `ptkdb` should own as application/UI state," because that boundary is the key design issue here.

Thought for 4s >

Yes — your instinct is right.

`DB` should be the **thin adapter to Perl's debugger hooks**. `Devel::ptkdb` should be the **debugger application**.

Perl requires certain things to exist in package `DB`, especially `DB::DB` and `DB::sub`; `DB::sub` specifically must be compiled in package `DB`. [Benin School of...](#) Perl's debugger machinery also uses the `DB` package for debugger state. [www-f9.ij...](#)

A good breakdown would be:

<> Perl



```
package DB;

# Required Perl debugger hook
sub DB {
    $DB::window->statement_hook($DB::filename, $DB::lineno);
}

# Required Perl debugger hook
sub sub {
    $DB::window->subroutine_hook($DB::sub);
}
```

Then most real logic belongs here:

<> Perl



```
package Devel::ptkdb;

sub statement_hook      { ... }
sub subroutine_hook     { ... }
sub initialize_gui      { ... }
sub update_source_window { ... }
sub handle_breakpoints  { ... }
sub step_into           { ... }
sub step_over           { ... }
sub continue_execution  { ... }
sub inspect_variable    { ... }
```

</> Perl



```
$DB::window = new Devel::ptkdb;
```

is not *wrong*, but it is architecturally ugly. It means: “store the GUI debugger object in the package Perl calls back into.” That was common old-school Perl style, but it does blur the boundary badly.

Better would be something like:

</> Perl



```
package DB;

our $debugger;

sub Initialize {
    $debugger ||= Devel::ptkdb->new;
}
```

and then:

</> Perl



```
sub DB {
    Initialize($DB::filename);
    $debugger->at_statement($DB::filename, $DB::lineno);
}
```

My rule of thumb:

`DB` package should contain only:

Perl-mandated debugger entry points
minimal glue code
access to Perl debugger globals
translation from Perl internals into method calls



`Devel::ptkdb` should contain:

GUI construction
Tk widgets
breakpoint management
stepping policy
stack display
variable inspection



```
user commands
ptkdb-specific state
```

For variables like:

</> Perl



```
$DB::on
$DB::subroutine_depth
$DB::step_over_depth
```

I would move them into the object unless Perl itself needs them in `DB`:

</> Perl



```
$self->{on}
$self->{subroutine_depth}
$self->{step_over_depth}
```

Keep in `DB` only variables that Perl itself expects, such as `$DB::single`, `$DB::trace`, `$DB::signal`, `$DB::sub`, `%DB::sub`, and the callback subs.

So the clean mental model is:

```
DB                = Perl debugger API boundary
Devel::ptkdb      = actual debugger implementation
$DB::window       = legacy global pointer to the implementation object
```



I would not try to eliminate `DB`, but I would make it as small and boring as possible.

I have completely isolated handling the state file load and save in the `Devel::ptkdb::State` module. I want to now replace the widget created by the sub `simplePromptBox` with `TK::JFileDialog`.

I'll look at the current `State` and UI code so the replacement hooks into your actual save/restore flow, not the older `simplePromptBox` design.

Thought for 58s >

Yes. I'd replace `simplePromptBox` with **one private filename chooser method** in `Devel::ptkdb::State`, then keep `save_state_callback` / `restore_state_callback` as thin wrappers.

One caution: Perl package names are case-sensitive. It would be `Tk::JFileDialog`, not `TK::JFileDialog`. Also, unless you specifically need `Tk::JFileDialog`, Perl/Tk already supports `getOpenFile` / `getSaveFile`; `Tk::FBox` is the Unix/X11 implementation behind those calls.  MetaCPAN... +1

I'd do this first, using built-in Perl/Tk dialogs:

<> Perl



```
use File::Basename qw(dirname basename);
```

Add this to `Devel::ptkdb::State`:

<> Perl



```
sub choose_state_file {
    my ($self, %args) = @_;

    my $debugger = $self->{debugger};
    my $mw       = $debugger->{main_window};

    my $current = $args{initial} || $self->state_file_name();

    my $dir = dirname($current);
    my $file = basename($current);

    my %opts = (
        -title      => $args{title} || 'Select ptkdb state file',
        -initialdir  => $dir,
        -initialfile => $file,
        -defaultextension => '.ptkdb',
        -filetypes => [
            ['ptkdb state files', '.ptkdb'],
            ['All files', '*'],
        ],
    );
};
```

```

        ? $mw->getSaveFile(%opts)
        : $mw->getOpenFile(%opts);

    return unless defined $chosen && length $chosen;

    $self->state_file_name($chosen);
    return $chosen;
}

```

Then replace `save_state_callback` with:

</> Perl



```

sub save_state_callback {
    my ($self, $name_in) = @_;

    my $debugger = $self->{debugger};

    my $chosen = $self->choose_state_file(
        mode    => 'save',
        title   => 'Save ptkdb state',
        initial => $name_in || $self->state_file_name(),
    );

    return unless defined $chosen;

    eval { $self->save_state_file($chosen) };
    $debugger->DoAlert($@) if $@;
}

```

And replace `restore_state_callback` with:

</> Perl



```

sub restore_state_callback {
    my ($self, $name_in) = @_;

    my $debugger = $self->{debugger};

    my $chosen = $self->choose_state_file(
        mode    => 'open',
        title   => 'Restore ptkdb state',
        initial => $name_in || $self->state_file_name(),
    );

    return unless defined $chosen;

    eval { $self->restore_state_file($chosen) };
    $debugger->DoAlert($@) if $@;
}

```

Then delete `simplePromptBox` from `State.pm`.

This preserves your architecture nicely:

`State.pm` owns state-file selection, save, restore
`ptkdb.pm` owns the menu entries
`DBHook.pm` owns DB breakpoint internals only



Please reiterate the steps needed for adding a debugger command line to my code. I want to initially support the commands s step in n step over r return l n goto line 'n' of currently displayed file I would like to put as much of this code as possible in a new module Devel::ptkdb::Cmd. Each block of code that you suggest must identify explicitly which file of this project it should be placed.

I didn't get useful hits from the file index, so I'm going straight against the uploaded files in `/mnt/data` and will keep the recommendations tied to those filenames.

Thought for 20s >

Below is the clean way I'd add it.

1. New file: `lib/Devel/ptkdb/Cmd.pm`

<> Perl



```
package Devel::ptkdb::Cmd;

use strict;
use warnings;

sub new {
    my ($class, %args) = @_;

    die "window is required" unless $args{window};

    return bless {
        window => $args{window},
    }, $class;
}

sub execute {
    my ($self, $command) = @_;

    $command //= '';
    $command =~ s/^\s+|\s+$//g;

    return unless length $command;

    my $window = $self->{window};

    if ($command eq 's') {
        return $window->{shared_callbacks}->{stepInSub}->();
    }

    if ($command eq 'n') {
        return $window->{shared_callbacks}->{stepOverSub}->();
    }

    if ($command eq 'r') {
        return $window->{shared_callbacks}->{returnSub}->();
    }

    if ($command =~ /^!\s+(\d+)\s/) {
```

```

    }

    Devel::ptkdb::console_say("Unknown debugger command: $command");
    return;
}

1;

```

2. File: lib/Devel/ptkdb.pm

Add this near the existing package-module imports:

</> Perl



```

use Devel::ptkdb::State;
use Devel::ptkdb::Cmd;

```

3. File: lib/Devel/ptkdb.pm

In `setup_main_window`, after:

</> Perl



```

$self->setup_button_bar();

```

add:

</> Perl



```

$self->setup_command_line();

```

So that section becomes:

</> Perl



```

# Button bar
$self->setup_button_bar();

# Debugger command line
$self->setup_command_line();

```

4. File: lib/Devel/ptkdb.pm

Add this new method near `setup_button_bar`:

</> Perl



```

sub setup_command_line {
    my ($self) = @_;

    my $mw = $self->{main_window};

    $self->{debugger_command} = Devel::ptkdb::Cmd->new(
        window => $self,
    );

    my $frm = $mw->Frame()->pack(
        -side => 'top',
        -fill => 'x',
    );

    $frm->Label(
        -text => 'Command:',
    )->pack(-side => 'left');

    my $entry = $frm->Entry(
        -width => 40,
    )->pack(
        -side => 'left',
        -fill => 'x',
        -expand => 1,
    );

    my $run_command = sub {
        my $command = $entry->get();

        $entry->delete(0, 'end');

        $self->{debugger_command}->execute($command);
    };

    $entry->bind('<Return>' => $run_command);

    $frm->Button(
        -text => 'Do',
        -command => $run_command,
    )->pack(-side => 'left');

    $self->{debugger_command_entry} = $entry;

    return;
}

```

5. File: `lib/Devel/ptkdb.pm`

Add this method near `DoGoto`:

<> Perl



```
sub goto_code_line {
    my ($self, $line) = @_;

    return unless defined $line;

    $line =~ s/^\s+|\s+$//g;
    return unless $line =~ /\d+$/;

    my $text_line = $line - $self->{'line_offset'};

    return if $text_line < 1;

    $self->{'text'}->see("$text_line.0");
    $self->{'text'}->markSet('insert', "$text_line.0");
    $self->{'text'}->focus();

    return 1;
}
```

6. File: lib/Devel/ptkdb.pm

Update `DoGoto` to reuse the same method:

<> Perl



```
sub DoGoto {
    my ($self, $entry) = @_;

    my $txt = $entry->get();

    $txt =~ s/^\s*(\d+).*/$1/;

    unless ($self->goto_code_line($txt)) {
        print "invalid text range\n";
        return;
    }

    $entry->selectionRange(0, 'end') if $entry->can('selectionRange');

    return;
}
```

The important idea: `Cmd.pm` should parse commands and dispatch them. The actual debugger/window behavior stays in `ptkdb.pm`, using your existing shared callbacks for `s`, `n`, and `r`, plus a new reusable `goto_code_line()` for `l n`.